

Generative Models in Finance

Week 4: Flow Matching and Diffusion Models — Foundations

Cristopher Salvi
Imperial College London

Spring 2026

Overview

- 1 Generative Modelling as Sampling
- 2 Flow Models
- 3 Diffusion Models
- 4 Constructing the Training Target
- 5 Training the Generative Model
- 6 A Guide to the Diffusion Model Literature
- 7 Building an Image Generator

Reference: P. Holderrieth & E. Erives, *An Introduction to Flow Matching and Diffusion Models*, MIT 6.S184, arXiv:2506.02070, 2025.

Part 1: Generative Modelling as Sampling

- The diversity of valid objects is modelled by a **data distribution** p_{data} over $\mathbb{R}^d$ (running assumption in these slides)
- **Generation = Sampling**: generating z means sampling $z \sim p_{\text{data}}$
- **Dataset**: i.i.d. samples $z_1, \dots, z_N \sim p_{\text{data}}$
- **Conditional generation**: sample $z \sim p_{\text{data}}(\cdot \mid y)$ for a conditioning variable y (e.g. text prompt). We focus on unconditional generation first
- **Goal**: Learn to transform samples from $p_{\text{init}} = \mathcal{N}(0, I_d)$ into samples from p_{data} , via simulation of a suitably constructed *differential equation*

Part 2: Flow Models — ODEs

- A **trajectory** is a function $X : [0, 1] \rightarrow \mathbb{R}^d$, $t \mapsto X_t$
- A **vector field** is a function

$$u : \mathbb{R}^d \times [0, 1] \rightarrow \mathbb{R}^d, \quad (x, t) \mapsto u_t(x),$$

specifying a velocity at every point x and time t

- An **ordinary differential equation (ODE)** with initial condition is:

$$\frac{d}{dt} X_t = u_t(X_t), \tag{4.1}$$

$$X_0 = x_0. \tag{4.2}$$

- Equation (4.1) says: the instantaneous rate of change of X_t is prescribed by the vector field u evaluated at the current position X_t

Flows

- The **flow** of the ODE is the function

$$\psi : \mathbb{R}^d \times [0, 1] \rightarrow \mathbb{R}^d, \quad (x_0, t) \mapsto \psi_t(x_0)$$

that maps each initial condition x_0 to the solution $X_t = \psi_t(x_0)$ at time t

- The flow satisfies:

$$\frac{d}{dt} \psi_t(x_0) = u_t(\psi_t(x_0)), \tag{4.3}$$

$$\psi_0(x_0) = x_0. \tag{4.4}$$

- **Intuition:** Vector fields, ODEs, and flows are three descriptions of the same object: *vector fields define ODEs whose solutions are flows*

Existence and Uniqueness

Theorem 4.1 (Flow existence and uniqueness)

If $u : \mathbb{R}^d \times [0, 1] \rightarrow \mathbb{R}^d$ is continuously differentiable with bounded derivative, then the ODE (4.1)–(4.2) has a unique solution given by a flow ψ_t . Moreover, ψ_t is a *diffeomorphism* for all t , i.e. ψ_t is continuously differentiable with continuously differentiable inverse ψ_t^{-1} .

- Since we parameterise $u_t(x)$ with neural networks with smooth activations (e.g. GELU, SiLU), the assumptions are approximately satisfied in practice
- **Takeaway:** Flows exist, are unique, and are diffeomorphisms — they “warp” space in an invertible way

Example 4.2 (Linear vector field)

Let $u_t(x) = -\theta x$ for $\theta > 0$. Then the flow is

$$\psi_t(x_0) = \exp(-\theta t) x_0. \quad (4.5)$$

Simulating an ODE: ODE Solvers

- In general, ψ_t cannot be computed in closed form; we approximate it numerically
- Write $t_k = kh$ for $k = 0, \dots, n$ with step size $h = n^{-1}$, and let $X_{t_0} = x_0$
- **Euler method** (order 1): one evaluation of u per step

$$X_{t_{k+1}} = X_{t_k} + h u_{t_k}(X_{t_k}) \quad (4.6)$$

- The **order** of a solver measures accuracy: an order- p method has global error $\mathcal{O}(h^p)$
- Higher-order methods use *multiple evaluations* of u per step to achieve better accuracy for the same step size

ODE Solvers: Higher-Order Methods

- **Midpoint method** (order 2): evaluate u at the midpoint

$$X_{t_k+h/2} = X_{t_k} + \frac{h}{2} u_{t_k}(X_{t_k})$$

► half-step

$$X_{t_{k+1}} = X_{t_k} + h u_{t_k+h/2}(X_{t_k+h/2})$$

► full step

- **Heun's method** (order 2): predictor-corrector strategy

$$\tilde{X}_{t_{k+1}} = X_{t_k} + h u_{t_k}(X_{t_k})$$

► predictor (Euler)

$$X_{t_{k+1}} = X_{t_k} + \frac{h}{2} (u_{t_k}(X_{t_k}) + u_{t_{k+1}}(\tilde{X}_{t_{k+1}}))$$

► corrector

- Both use **2 evaluations** of u per step; same order, different geometric properties

ODE Solvers: Runge–Kutta Methods

- The classical 4th-order Runge–Kutta method (RK4):

$$k_1 = u_{t_k}(X_{t_k}),$$

$$k_2 = u_{t_k+h/2}(X_{t_k} + \frac{h}{2} k_1),$$

$$k_3 = u_{t_k+h/2}(X_{t_k} + \frac{h}{2} k_2),$$

$$k_4 = u_{t_{k+1}}(X_{t_k} + h k_3),$$

$$X_{t_{k+1}} = X_{t_k} + \frac{h}{6} (k_1 + 2k_2 + 2k_3 + k_4)$$

- Uses 4 evaluations of u per step; global error $\mathcal{O}(h^4)$
- Euler, midpoint, and Heun are all special cases of the general family of Runge–Kutta methods, characterised by their Butcher tableau

ODE Solvers: Practical Considerations

- **Cost–accuracy trade-off:** each step of an order- p method requires $s \geq p$ evaluations of u , but fewer steps are needed for the same accuracy
- **Adaptive step-size control:** modern solvers (e.g. Dormand–Prince / `dopri5`) *adapt* h at each step by comparing two estimates of different orders
- For the rest of these lectures, we use the Euler method for simplicity, but all results generalise to higher-order solvers

Flow Models

- A **flow model** is defined by:

$$X_0 \sim p_{\text{init}}$$

► random initialisation

$$\frac{d}{dt}X_t = u_t^\theta(X_t)$$

► ODE

where $u_t^\theta : \mathbb{R}^d \times [0, 1] \rightarrow \mathbb{R}^d$ is a neural network with parameters θ

- **Goal:** Choose θ so that the endpoint X_1 has distribution p_{data} , i.e.

$$X_1 \sim p_{\text{data}} \quad \Leftrightarrow \quad \psi_1^\theta(X_0) \sim p_{\text{data}}$$

- **Important:** The neural network parameterises the *vector field*, not the flow itself. The flow is obtained by simulating the ODE

Sampling from a Flow Model

Algorithm 1: Sampling from a Flow Model (Euler method)

Require: Neural network vector field u_t^θ , number of steps n

1. Set $t = 0$, step size $h = 1/n$
2. Draw $X_0 \sim p_{\text{init}}$
3. For $i = 1, \dots, n$ do:
4. $X_{t+h} = X_t + h u_t^\theta(X_t)$
5. $t \leftarrow t + h$
6. Return X_1

Part 3: Diffusion Models — Brownian Motion

Definition 4.3 (Brownian motion)

A *Brownian motion* (or *Wiener process*) $W = (W_t)_{0 \leq t \leq 1}$ is a stochastic process such that $W_0 = 0$, the trajectories $t \mapsto W_t$ are continuous, and:

- ① *Normal increments:* $W_t - W_s \sim \mathcal{N}(0, (t-s)I_d)$ for all $0 \leq s < t$
- ② *Independent increments:* For any $0 \leq t_0 < t_1 < \dots < t_n$, the increments $W_{t_1} - W_{t_0}, \dots, W_{t_n} - W_{t_{n-1}}$ are independent

- Approximate simulation: set $W_0 = 0$ and update

$$W_{t+h} = W_t + \sqrt{h} \epsilon_t, \quad \epsilon_t \sim \mathcal{N}(0, I_d) \quad (4.7)$$

- Brownian paths are continuous but nowhere differentiable (a.s.)
- Brownian motion is the fundamental building block of stochastic calculus, analogous to the Gaussian distribution in probability theory

Advanced BM Simulation: Brownian Bridge and Virtual Brownian Tree

- The sequential scheme (4.7) generates W on a *fixed* grid; adaptive solvers may query W at *arbitrary* times
- **Brownian bridge interpolation:** Given W_s and W_t , for any $s < r < t$:

$$W_r \mid W_s, W_t \sim \mathcal{N}\left(\frac{(t-r)W_s + (r-s)W_t}{t-s}, \frac{(r-s)(t-r)}{t-s} I_d\right)$$

This allows generating BM values at new times *conditionally* on already-sampled values

- **Virtual Brownian Tree**¹: organise time intervals in a *binary tree*; at each query time r , walk down the tree and apply Brownian bridge interpolation
- The entire path is determined by a **single PRNG seed**: previously generated values need not be stored $\Rightarrow \mathcal{O}(1)$ memory, $\mathcal{O}(\log(1/\text{tol}))$ time per query
- This enables *adaptive step-size* SDE solvers: the solver chooses h on the fly while maintaining a consistent Brownian path across rejected/accepted steps
- Implementation available in **DiffraX**², a JAX library for numerical differential equation solvers

¹Kidger, *On Neural Differential Equations*, PhD thesis, Oxford, 2021. Extended by Jelinčič, Foster & Kidger, *Single-seed generation of Brownian paths and integrals for adaptive and high order SDE solvers*, arXiv:2405.06464, 2024.

²<https://github.com/patrick-kidger/diffrax>

From ODEs to SDEs

- An ODE $dX_t = u_t(X_t) dt$ defines a *deterministic* flow; given $X_0 = x_0$, the trajectory is uniquely determined
- An **Itô stochastic differential equation** perturbs this with a martingale driving term:

$$dX_t = u_t(X_t) dt + \sigma_t dW_t, \quad (4.8)$$

$$X_0 = x_0, \quad (4.9)$$

where $\sigma_t \geq 0$ is the **diffusion coefficient** and W is a d -dimensional Brownian motion

- The integral form makes the meaning precise: for all $0 \leq s < t \leq 1$,

$$X_t = X_s + \int_s^t u_r(X_r) dr + \int_s^t \sigma_r dW_r \quad (4.10)$$

where the second integral is an **Itô integral** (defined as an L^2 -limit of Riemann sums against W)

- **Key distinction from ODEs:** The solution (X_t) is now a *stochastic process*, not a deterministic path. There is no flow map ψ_t ; instead, X_t defines a random variable for each t , adapted to the filtration generated by W

SDE Existence and Uniqueness

Theorem 4.4 (Strong existence and pathwise uniqueness^a)

^aMao, X. *Stochastic Differential Equations and Applications*, 2nd ed., Horwood, 2007, Ch. 3.

Suppose that:

- ① (*Lipschitz continuity*) There exists $L > 0$ such that for all $x, y \in \mathbb{R}^d$ and $t \in [0, 1]$:

$$\|u_t(x) - u_t(y)\| \leq L\|x - y\|$$

- ② (*Linear growth*) There exists $K > 0$ such that for all $x \in \mathbb{R}^d$ and $t \in [0, 1]$:

$$\|u_t(x)\|^2 + |\sigma_t|^2 \leq K^2(1 + \|x\|^2)$$

Then the SDE (4.8)–(4.9) admits a unique *strong solution*: a continuous, $(\mathcal{F}_t^W)$ -adapted process $(X_t)_{0 \leq t \leq 1}$ satisfying (4.10) a.s., with $\mathbb{E}[\sup_{0 \leq t \leq 1} \|X_t\|^2] < \infty$.

- The proof proceeds by *Picard iteration* on the integral equation (4.10), using the Itô isometry and the Borel–Cantelli lemma to upgrade L^2 -convergence to a.s. uniform convergence
- Every ODE is a special case with $\sigma_t = 0$. Henceforth, *when we speak about SDEs, we consider ODEs as a special case*

The Ornstein–Uhlenbeck Process

Example 4.5 (Ornstein–Uhlenbeck process)

Let $u_t(x) = -\theta x$ for $\theta > 0$ and $\sigma_t = \sigma \geq 0$ constant. The SDE

$$dX_t = -\theta X_t dt + \sigma dW_t \quad (4.11)$$

defines an *Ornstein–Uhlenbeck (OU) process*.

- The drift is globally Lipschitz with $L = \theta$, so the theorem applies
- **Closed-form solution** (by variation of constants / integrating factor $e^{\theta t}$):

$$X_t = e^{-\theta t} x_0 + \sigma \int_0^t e^{-\theta(t-s)} dW_s$$

- Since the Itô integral of a deterministic integrand is Gaussian:

$$X_t \sim \mathcal{N}\left(e^{-\theta t} x_0, \frac{\sigma^2}{2\theta}(1 - e^{-2\theta t}) I_d\right)$$

- As $t \rightarrow \infty$: $X_t \xrightarrow{d} \mathcal{N}(0, \frac{\sigma^2}{2\theta} I_d)$ (stationary distribution)
- For $\sigma = 0$: reduces to the deterministic flow $\psi_t(x_0) = e^{-\theta t} x_0$ from (4.5)

SDE Solvers: Euler–Maruyama Method

- The **Euler–Maruyama method**³ is the SDE analogue of the Euler method for ODEs:

$$X_{t_{k+1}} = X_{t_k} + h u_{t_k}(X_{t_k}) + \sigma_{t_k}(W_{t_{k+1}} - W_{t_k}) \quad (4.12)$$

Since $W_{t_{k+1}} - W_{t_k} \sim \mathcal{N}(0, h I_d)$, equivalently: $X_{t_{k+1}} = X_{t_k} + h u_{t_k}(X_{t_k}) + \sqrt{h} \sigma_{t_k} \epsilon_k$, $\epsilon_k \sim \mathcal{N}(0, I_d)$

- When $\sigma_t = 0$, this reduces to the Euler method (4.6)
- **Two notions of convergence** for SDE solvers:
 - ▶ **Strong order p** : $\mathbb{E}[\|X_T - X_T^h\|] = \mathcal{O}(h^p)$ (pathwise accuracy)
 - ▶ **Weak order q** : $|\mathbb{E}[f(X_T)] - \mathbb{E}[f(X_T^h)]| = \mathcal{O}(h^q)$ for smooth f (distributional accuracy)
- Euler–Maruyama has **strong order 1/2** and **weak order 1**

³Maruyama, G. *Continuous Markov processes and stochastic equations*, Rend. Circ. Mat. Palermo, 4:48–90, 1955.

SDE Solvers: Milstein Method

- Euler–Maruyama ignores the interaction between noise and drift. The **Milstein method**⁴ adds a correction term from the Itô–Taylor expansion:

$$X_{t_{k+1}} = X_{t_k} + h u_{t_k}(X_{t_k}) + \sigma_{t_k} \Delta W_k + \frac{1}{2} \sigma_{t_k} \sigma'_{t_k} ((\Delta W_k)^2 - h I_d)$$

where $\Delta W_k = W_{t_{k+1}} - W_{t_k}$ and $\sigma'_{t_k} = \frac{d\sigma}{dt} \Big|_{t_k}$

- This achieves **strong order 1** (and weak order 1), matching the ODE Euler method's convergence rate
- **Caveat (multidimensional case):** When $d > 1$ and the diffusion coefficient depends on x , the Milstein correction requires simulating **Lévy areas** $\int_{t_k}^{t_{k+1}} (W_s^i - W_{t_k}^i) dW_s^j$, which is non-trivial
- For scalar noise ($d = 1$) or additive noise (σ_t independent of x), Milstein is straightforward and recommended over Euler–Maruyama

⁴Milstein, G.N. *Approximate integration of stochastic differential equations*, Theory Probab. Appl., 19(3):557–562, 1974.

SDE Solvers: Higher-Order and Adaptive Methods

- **Stochastic Runge–Kutta methods**⁵: Achieve strong order 1 (or higher) *without* explicit derivatives of u or σ , by evaluating the coefficients at multiple intermediate points — analogous to deterministic RK methods
- **Key difficulty vs. ODEs**: Achieving strong order > 1 requires simulating iterated stochastic integrals (Lévy areas), which adds computational cost; most practical SDE solvers therefore operate at strong order ≤ 1
- **Adaptive step-size control**: Possible via the Virtual Brownian Tree (cf. earlier slide); reject steps where the local error estimate exceeds a tolerance, and retry with smaller h
- For *weak* approximation (sufficient for sampling in generative models), higher weak-order methods exist that avoid Lévy areas entirely

General reference: Kloeden, P.E. & Platen, E. *Numerical Solution of Stochastic Differential Equations*, Springer, 1992.

⁵Rößler, A. *Runge–Kutta methods for the strong approximation of solutions of SDEs*, SIAM J. Numer. Anal., 48(3):922–952, 2010.

The Rough Path Perspective on SDEs

- **Core problem:** The Itô integral $\int_s^t f(X_r) dW_r$ is defined via L^2 -limits. This *probabilistic* construction obscures the *analytic* structure needed for high-order numerics
- **Lyons' rough paths theory**⁶: Reformulate the SDE as a *pathwise* differential equation driven by the Brownian motion viewed as a **rough path**
- A rough path over W is not just the path $t \mapsto W_t$, but an **enhanced path** that also encodes iterated integrals:

$$\mathbf{W}_t = \left(W_t, \mathbb{W}_{s,t} := \int_s^t (W_r - W_s) \otimes \circ dW_r \right)$$

The second component $\mathbb{W}_{s,t} \in \mathbb{R}^{d \times d}$ is the **Lévy area** (the antisymmetric part encodes the “winding” of the path). The integral is in the **Stratonovich** sense

- **Key insight:** Once $\mathbf{W}$ is fixed, the solution map $\mathbf{W} \mapsto X$ is *continuous* in rough path topology — the **Universal Limit Theorem**⁷. This yields solutions in the **Stratonovich** sense; the Itô solution is recovered via the Itô–Stratonovich correction:

$$\int_s^t f(X_r) \circ dW_r = \int_s^t f(X_r) dW_r + \frac{1}{2} \int_s^t f'(X_r) \sigma_r dr$$

⁶Lyons, T.J. *Differential equations driven by rough signals*, Rev. Mat. Iberoam., 14(2):215–310, 1998.

⁷Friz, P.K. & Hairer, M. *A Course on Rough Paths*, 2nd ed., Springer, 2020, Ch. 8.

Rough Paths and Numerical Schemes

- The ULT implies: to approximate X , it suffices to approximate the *driving rough path* $\mathbf{W}$ well. Since the ULT operates in the Stratonovich setting, the numerical schemes below approximate the **Stratonovich SDE**

$$\circ dX_t = \tilde{u}_t(X_t) dt + \sigma_t \circ dW_t$$

where $\tilde{u}_t(x) = u_t(x) - \frac{1}{2}\sigma_t\sigma_t'$ absorbs the Itô–Stratonovich drift correction

- **Davie's scheme**⁸: Given the Brownian increment $\Delta W_k = W_{t_{k+1}} - W_{t_k}$ and the Lévy area $\mathbb{W}_{t_k, t_{k+1}}$, update via

$$X_{t_{k+1}} = X_{t_k} + \tilde{u}_{t_k}(X_{t_k}) h + \sigma_{t_k} \Delta W_k + D\sigma_{t_k}(X_{t_k}) \sigma_{t_k} \mathbb{W}_{t_k, t_{k+1}}$$

- This achieves **strong order 1** and makes explicit *why* Lévy areas are needed: they are part of the rough path data at the required level of regularity
- The Milstein scheme (in Stratonovich form) is recovered as a special case

⁸Davie, A.M. *Differential equations driven by rough paths: an approach via discrete approximation*, Appl. Math. Res. Express, 2008, arXiv:0710.0772.

Log-ODE Method and Higher-Order Rough Path Solvers

- **Log-ODE method**⁹: On each interval $[t_k, t_{k+1}]$, replace the rough driving signal by a *smooth* path (a geodesic in the free nilpotent group) that matches the same iterated integrals, then solve the resulting ODE exactly or to high order
- This transforms the SDE solve into a sequence of *local ODE solves*, for which classical Runge–Kutta methods apply
- **Convergence**: Approximating the rough path signature to level N yields a method of strong order $N/2 - \varepsilon$. Level $N = 2$ (increments + Lévy area) gives strong order 1; level $N = 3$ adds further iterated integrals for strong order $3/2$
- **Dimension-free error bounds**¹⁰: Rough path estimates give error bounds that depend on the *regularity* of the driving signal, not the ambient dimension d — a crucial advantage for high-dimensional problems in generative modelling

General reference: Friz, P.K. & Victoir, N.B. *Multidimensional Stochastic Processes as Rough Paths*, Cambridge, 2010.

⁹Foster, J., Lyons, T. & Oberhauser, H. *An optimal polynomial approximation of Brownian motion*, SIAM J. Numer. Anal., 58(3):1393–1421, 2020, arXiv:1904.06998.

¹⁰Boutaib, Y., Gyurkó, L.G., Lyons, T. & Yang, D. *Dimension-free Euler estimates of rough differential equations*, Rev. Roumaine Math. Pures Appl., 59(1):25–53, 2014, arXiv:1307.4708.

Diffusion Models

- A **diffusion model** is defined by:

$$dX_t = u_t^\theta(X_t) dt + \sigma_t dW_t$$

$$X_0 \sim p_{\text{init}}$$

- ▶ SDE
- ▶ random initialisation

- It consists of:

- ▶ A neural network $u^\theta : \mathbb{R}^d \times [0, 1] \rightarrow \mathbb{R}^d$, $(x, t) \mapsto u_t^\theta(x)$ with parameters θ
- ▶ A fixed diffusion coefficient $\sigma_t : [0, 1] \rightarrow [0, \infty)$

- **Goal:** $X_1 \sim p_{\text{data}}$
- A diffusion model with $\sigma_t = 0$ is a **flow model**

Sampling from a Diffusion Model

Algorithm 2: Sampling from a Diffusion Model (Euler–Maruyama)

Require: Neural network u_t^θ , number of steps n , diffusion coefficient σ_t

1. Set $t = 0$, step size $h = 1/n$
2. Draw $X_0 \sim p_{\text{init}}$
3. **For** $i = 1, \dots, n$ **do:**
4. Draw $\epsilon \sim \mathcal{N}(0, I_d)$
5. $X_{t+h} = X_t + h u_t^\theta(X_t) + \sigma_t \sqrt{h} \epsilon$
6. $t \leftarrow t + h$
7. **Return** X_1

Part 4: Constructing the Training Target

- If we randomly initialise θ , simulating the ODE/SDE produces nonsense
- We need to *train* the neural network by minimising a loss:

$$\mathcal{L}(\theta) = \|u_t^\theta(x) - u_t^{\text{target}}(x)\|^2 \quad (4.13)$$

- The **training target** $u_t^{\text{target}} : \mathbb{R}^d \times [0, 1] \rightarrow \mathbb{R}^d$ is a vector field whose corresponding ODE/SDE converts p_{init} into p_{data}
- **This part:** Derive a formula for u_t^{target}
- **Next part:** Design a tractable training algorithm to approximate u_t^{target}

Conditional Probability Paths

Definition 4.6 (Conditional probability path)

A *conditional (interpolating) probability path* is a family of distributions $p_t(\cdot \mid z)$ over $\mathbb{R}^d$, parameterised by $t \in [0, 1]$ and $z \in \mathbb{R}^d$, such that

$$p_0(\cdot \mid z) = p_{\text{init}}, \quad p_1(\cdot \mid z) = \delta_z, \quad \text{for all } z \in \mathbb{R}^d. \quad (4.14)$$

- Here δ_z is the Dirac delta at z (the distribution that always returns z)
- Intuitively, $p_t(\cdot \mid z)$ gradually converts a *single* data point z from noise p_{init} at $t = 0$ to a point mass at z at $t = 1$
- Think of this as a trajectory in the space of distributions, indexed by t

Marginal Probability Paths

Definition 4.7 (Marginal probability path)

Given a conditional probability path $\{p_t(\cdot | z)\}_{t \in [0,1], z \in \mathbb{R}^d}$ and a data distribution p_{data} , the **marginal probability path** $\{p_t\}_{t \in [0,1]}$ is the mixture

$$p_t(x) = \int_{\mathbb{R}^d} p_t(x | z) p_{\text{data}}(z) dz = \mathbb{E}_{z \sim p_{\text{data}}}[p_t(x | z)]. \quad (4.15)$$

- **Equivalent sampling procedure:**

$$z \sim p_{\text{data}}, \quad x \sim p_t(\cdot | z) \quad \Rightarrow \quad x \sim p_t \quad (4.16)$$

- **Boundary conditions** (inherited from the conditional path):

$$p_0(x) = \int p_0(x | z) p_{\text{data}}(z) dz = \int p_{\text{init}}(x) p_{\text{data}}(z) dz = p_{\text{init}}(x), \quad (4.17)$$

$$p_1(x) = \int \delta_z(x) p_{\text{data}}(z) dz = p_{\text{data}}(x). \quad (4.18)$$

- **Computational note:** We can *sample* from p_t via (4.16), but we cannot *evaluate* $p_t(x)$ because the integral (4.15) is intractable (it marginalises over the entire data distribution)

Gaussian Conditional Probability Paths

Example 4.8 (Gaussian conditional probability path)

Let α_t, β_t be *noise schedulers*: continuously differentiable, monotonic functions with $\alpha_0 = \beta_1 = 0$ and $\alpha_1 = \beta_0 = 1$. Define

$$p_t(\cdot \mid z) = \mathcal{N}(\alpha_t z, \beta_t^2 I_d). \quad (4.19)$$

- Check: $p_0(\cdot \mid z) = \mathcal{N}(0, \beta_0^2 I_d) = \mathcal{N}(0, I_d) = p_{\text{init}} \checkmark$
- Check: $p_1(\cdot \mid z) = \mathcal{N}(\alpha_1 z, 0) = \delta_z \checkmark$
- Sampling from the marginal path:

$$z \sim p_{\text{data}}, \quad \epsilon \sim \mathcal{N}(0, I_d) \quad \Rightarrow \quad x = \alpha_t z + \beta_t \epsilon \sim p_t \quad (4.20)$$

- Intuitively: more noise for smaller t (closer to $t = 0$, pure noise), less noise for larger t (closer to $t = 1$, pure data)

Interlude: The Wasserstein-2 Distance

- **Optimal transport**¹¹ asks: what is the cheapest way to move mass from a distribution μ to a distribution ν , when transporting a unit of mass from x to y costs $\|x - y\|^2$?
- The **Wasserstein-2 distance** solves this by optimising over all **couplings** (joint distributions with the correct marginals):

$$W_2^2(\mu, \nu) = \inf_{\gamma \in \Pi(\mu, \nu)} \int_{\mathbb{R}^d \times \mathbb{R}^d} \|x - y\|^2 d\gamma(x, y),$$

where $\Pi(\mu, \nu) = \{\gamma \in \mathcal{P}(\mathbb{R}^d \times \mathbb{R}^d) : \gamma(\cdot, \mathbb{R}^d) = \mu, \gamma(\mathbb{R}^d, \cdot) = \nu\}$

- **Geodesic:** $(\mathcal{P}_2(\mathbb{R}^d), W_2)$ is a metric space¹². A **constant-speed geodesic** is a curve $(\mu_t)_{t \in [0,1]}$ satisfying

$$W_2(\mu_s, \mu_t) = |t - s| W_2(\mu_0, \mu_1), \quad \forall s, t \in [0, 1].$$

- **Displacement interpolation**¹³: given an optimal transport map T , the path $\mu_t = ((1 - t) \text{id} + t T)_{\#} \mu$ is the unique constant-speed geodesic from $\mu_0 = \mu$ to $\mu_1 = T_{\#} \mu = \nu$

¹¹Villani, C. (2009). *Optimal Transport: Old and New*. Springer. Grundlehren der mathematischen Wissenschaften, Vol. 338.

¹²Ambrosio, L., Gigli, N., Savaré, G. (2008). *Gradient Flows in Metric Spaces and in the Space of Probability Measures*. 2nd ed., Birkhäuser. Lectures in Mathematics ETH Zürich.

¹³McCann, R.J. (1997). A convexity principle for interacting gases. *Advances in Mathematics*, 128(1), 153–179.

The Conditional Optimal Transport Path

Set $\alpha_t = t$, $\beta_t = 1 - t$ ¹⁴:

$$p_t(\cdot \mid z) = \mathcal{N}(t z, (1 - t)^2 I_d), \quad X_t = t z + (1 - t) \epsilon, \quad \epsilon \sim \mathcal{N}(0, I_d). \quad (4.21)$$

- The unique OT map from $\mathcal{N}(0, I_d)$ to δ_z is $T(x) = z$ (all mass goes to z)
- The displacement interpolation $T_t(x) = (1 - t)x + t z$ recovers exactly (4.21)
- Hence the CondOT path is the W_2 -geodesic from $\mathcal{N}(0, I_d)$ to δ_z
- **Key consequence:** Each sample follows a *straight line* with constant velocity:

$$\dot{X}_t = z - \epsilon$$

Straighter trajectories $\Rightarrow$ fewer ODE discretisation steps at inference

¹⁴Lipman, Y., Chen, R.T.Q., Ben-Hamu, H., Nickel, M. (2023). Flow Matching for Generative Modeling. *ICLR 2023*. arXiv:2210.02747.

Conditional Vector Fields

- We now construct a vector field whose ODE trajectories follow the conditional probability path
- A **conditional vector field** $u_t^{\text{target}}(\cdot | z)$ is defined so that its ODE yields $X_t \sim p_t(\cdot | z)$:

$$X_0 \sim p_{\text{init}}, \quad \frac{d}{dt}X_t = u_t^{\text{target}}(X_t | z) \quad \Rightarrow \quad X_t \sim p_t(\cdot | z). \quad (4.22)$$

Conditional Gaussian Vector Field

Example 4.9 (Target ODE for Gaussian probability paths)

For $p_t(\cdot | z) = \mathcal{N}(\alpha_t z, \beta_t^2 I_d)$, the *conditional Gaussian vector field* is

$$u_t^{\text{target}}(x | z) = \left(\dot{\alpha}_t - \frac{\dot{\beta}_t}{\beta_t} \alpha_t \right) z + \frac{\dot{\beta}_t}{\beta_t} x. \quad (4.23)$$

Proof. Define the map $\psi_t^{\text{target}}(x | z) = \alpha_t z + \beta_t x$. This is a valid flow:

- $\psi_0^{\text{target}}(x | z) = \alpha_0 z + \beta_0 x = 0 + 1 \cdot x = x$ ($\psi_0 = \text{id}$) ✓
- For $t \in [0, 1]$, $\beta_t > 0$, so $\psi_t^{\text{target}}(\cdot | z)$ is a diffeomorphism with inverse $(\psi_t^{\text{target}})^{-1}(y | z) = (y - \alpha_t z) / \beta_t$ ✓

If $X_0 \sim \mathcal{N}(0, I_d) = p_{\text{init}}$, then by the affine property of Gaussians:

$$X_t = \psi_t^{\text{target}}(X_0 | z) = \alpha_t z + \beta_t X_0 \sim \mathcal{N}(\alpha_t z, \beta_t^2 I_d) = p_t(\cdot | z). \quad (4.24)$$

Conditional Gaussian Vector Field (Proof cont.)

It remains to extract $u_t^{\text{target}}(x | z)$ from ψ_t^{target} . By definition of a flow (4.3):

$$\frac{d}{dt} \psi_t^{\text{target}}(x | z) = u_t^{\text{target}}(\psi_t^{\text{target}}(x | z) | z) \quad \text{for all } x, z \in \mathbb{R}^d.$$

Differentiating $\psi_t^{\text{target}}(x | z) = \alpha_t z + \beta_t x$ with respect to t :

$$\dot{\alpha}_t z + \dot{\beta}_t x = u_t^{\text{target}}(\alpha_t z + \beta_t x | z).$$

Since $\beta_t > 0$ for $t \in [0, 1)$, the map $x \mapsto \alpha_t z + \beta_t x$ is surjective onto $\mathbb{R}^d$. Substituting $x = (\psi_t^{\text{target}})^{-1}(y | z) = (y - \alpha_t z) / \beta_t$:

$$\dot{\alpha}_t z + \dot{\beta}_t \cdot \frac{y - \alpha_t z}{\beta_t} = u_t^{\text{target}}(y | z) \quad \text{for all } y \in \mathbb{R}^d.$$

Rearranging (collecting the z and y terms):

$$u_t^{\text{target}}(y | z) = \left(\dot{\alpha}_t - \frac{\dot{\beta}_t}{\beta_t} \alpha_t \right) z + \frac{\dot{\beta}_t}{\beta_t} y. \quad \square$$

The Marginalization Trick

Theorem 4.10 (Marginalization trick)

Let $u_t^{\text{target}}(\cdot | z)$ be a conditional vector field satisfying (4.22). Then the *marginal vector field*

$$u_t^{\text{target}}(x) = \int u_t^{\text{target}}(x | z) \frac{p_t(x | z) p_{\text{data}}(z)}{p_t(x)} dz \quad (4.25)$$

yields ODE trajectories that follow the *marginal* probability path:

$$X_0 \sim p_{\text{init}}, \quad \frac{d}{dt} X_t = u_t^{\text{target}}(X_t) \quad \Rightarrow \quad X_t \sim p_t \quad (0 \leq t \leq 1). \quad (4.26)$$

In particular, $X_1 \sim p_{\text{data}}$.

- **Key insight:** We can construct the (intractable) marginal vector field from the (tractable) conditional vector field via (4.25)
- The proof uses the *continuity equation*

The Continuity Equation

Theorem 4.11 (Continuity equation)

Let u_t^{target} be a vector field with $X_0 \sim p_{\text{init}}$. Then $X_t \sim p_t$ for all $0 \leq t \leq 1$ if and only if

$$\partial_t p_t(x) = -\text{div}(p_t u_t^{\text{target}})(x) \quad \text{for all } x \in \mathbb{R}^d, \quad 0 \leq t \leq 1, \quad (4.27)$$

where $\text{div}(v_t)(x) = \sum_{i=1}^d \frac{\partial}{\partial x_i} v_t^i(x)$ is the *divergence* operator.

- **Interpretation:** The left side $\partial_t p_t(x)$ measures how probability density changes at x . The right side $-\text{div}(p_t u_t)$ measures the net inflow of probability mass via the vector field. Since mass is conserved, these must be equal
- This is a special case ($\sigma_t = 0$) of the Fokker–Planck equation (Theorem 4.14), whose proof is given later. We use this result to prove the Marginalization Trick

Proof of the Marginalization Trick

We show that $u_t^{\text{target}}(x)$ defined in (4.25) satisfies the continuity equation (4.27).

$$\begin{aligned}\partial_t p_t(x) &\stackrel{(i)}{=} \partial_t \int p_t(x | z) p_{\text{data}}(z) dz \\ &= \int \partial_t p_t(x | z) p_{\text{data}}(z) dz \\ &\stackrel{(ii)}{=} \int -\text{div}(p_t(\cdot | z) u_t^{\text{target}}(\cdot | z))(x) p_{\text{data}}(z) dz \\ &\stackrel{(iii)}{=} -\text{div} \left(\int p_t(x | z) u_t^{\text{target}}(x | z) p_{\text{data}}(z) dz \right)\end{aligned}$$

where:

- (i) definition of $p_t(x)$ as $\int p_t(x | z) p_{\text{data}}(z) dz$ (equation (4.15));
- (ii) the conditional vector field satisfies the continuity equation:
 $\partial_t p_t(\cdot | z) = -\text{div}(p_t(\cdot | z) u_t^{\text{target}}(\cdot | z))$ (this holds by hypothesis (4.22) and Theorem 4.11 applied to each fixed z);
- (iii) interchange of div_x and $\int dz$ (justified by Fubini–Tonelli, using the regularity of u_t^{target} and p_t). Passing ∂_t under the integral from line 1 to line 2 is similarly justified by dominated convergence.

Proof of the Marginalization Trick (cont.)

Continuing:

$$\begin{aligned} &\stackrel{(iv)}{=} -\operatorname{div} \left(p_t(x) \int u_t^{\text{target}}(x | z) \frac{p_t(x | z) p_{\text{data}}(z)}{p_t(x)} dz \right) \\ &\stackrel{(v)}{=} -\operatorname{div}(p_t u_t^{\text{target}})(x), \end{aligned}$$

where:

- (iv) factor out $p_t(x) > 0$ (the marginal density is strictly positive on its support, since it is a mixture of Gaussians);
- (v) recognise the definition of $u_t^{\text{target}}(x)$ in (4.25).

The first and last expressions give $\partial_t p_t(x) = -\operatorname{div}(p_t u_t^{\text{target}})(x)$, which is the continuity equation (4.27) for the marginal vector field. By Theorem 4.11, this implies $X_t \sim p_t$ for $0 \leq t \leq 1$. □

The Score Function

Definition 4.12 (Score function)

The *marginal score function* of p_t is

$$\nabla \log p_t(x) = \frac{\nabla p_t(x)}{p_t(x)}.$$

The *conditional score function* of $p_t(\cdot | z)$ is $\nabla \log p_t(x | z)$.

- The marginal score can be expressed via conditional scores:

$$\nabla \log p_t(x) = \int \nabla \log p_t(x | z) \frac{p_t(x | z) p_{\text{data}}(z)}{p_t(x)} dz \quad (4.28)$$

- The conditional score is typically known analytically

Score Function for Gaussian Paths

Example 4.13 (Score for Gaussian conditional probability paths)

For $p_t(x | z) = \mathcal{N}(x; \alpha_t z, \beta_t^2 I_d)$, the conditional score is

$$\nabla \log p_t(x | z) = -\frac{x - \alpha_t z}{\beta_t^2}. \quad (4.29)$$

- **Proof.** The Gaussian density is $\mathcal{N}(x; \mu, \sigma^2 I_d) = (2\pi\sigma^2)^{-d/2} \exp\left(-\frac{\|x-\mu\|^2}{2\sigma^2}\right)$. Taking $\nabla_x \log$ gives $-\frac{x-\mu}{\sigma^2}$. With $\mu = \alpha_t z$ and $\sigma^2 = \beta_t^2$, the result follows
- **Key property:** The conditional score is *linear* in x . This is a special feature of Gaussian distributions

From ODEs to SDEs: Extending the Continuity Equation

- So far, we have derived the training target for **flow models** (ODEs):
 - ▶ Conditional vector field $u_t^{\text{target}}(x | z)$ generates the conditional path $p_t(\cdot | z)$
 - ▶ Marginalization trick (Theorem 4.10): the marginal vector field generates p_t
 - ▶ The proof relied on the **continuity equation** (4.27)
- **Goal:** Extend this to **diffusion models** (SDEs) with $\sigma_t > 0$
- **Problem:** The continuity equation only governs ODEs. For SDEs, we need to account for the additional spreading of probability mass caused by Brownian noise
- **Solution:** The **Fokker–Planck equation** — the SDE analogue of the continuity equation. It adds a second-order (diffusion) term $\frac{\sigma_t^2}{2} \Delta p_t$ to capture the effect of noise
- We will use the score function $\nabla \log p_t$ (just defined) to connect the Fokker–Planck equation back to a concrete SDE formula for diffusion models

The Fokker–Planck Equation

Theorem 4.14 (Fokker–Planck equation)

Let $u_t : \mathbb{R}^d \rightarrow \mathbb{R}^d$ be Lipschitz continuous and $\sigma_t \geq 0$ continuous. Consider the SDE

$$X_0 \sim p_{\text{init}}, \quad dX_t = u_t(X_t) dt + \sigma_t dW_t.$$

Suppose the law of X_t admits a smooth density p_t for each $t \in [0, 1]$. Then p_t satisfies the *Fokker–Planck equation*:

$$\partial_t p_t(x) = -\operatorname{div}(p_t u_t)(x) + \frac{\sigma_t^2}{2} \Delta p_t(x), \quad (4.30)$$

where $\Delta p_t(x) = \sum_{i=1}^d \frac{\partial^2}{\partial x_i^2} p_t(x)$ is the *Laplacian*.

Conversely, if a smooth probability density p_t satisfies (4.30) with $p_0 = p_{\text{init}}$, then $X_t \sim p_t$ for all t .

- For $\sigma_t = 0$: reduces to the continuity equation (4.27)
- The term $\frac{\sigma_t^2}{2} \Delta p_t$ captures the spreading of probability mass due to Brownian noise (cf. the heat equation $\partial_t p = \frac{\sigma^2}{2} \Delta p$, which is the special case $u_t = 0$)

Proof of the Fokker–Planck Equation (Necessity)

Goal: Show that the density p_t of X_t satisfies (4.30). The strategy is to test against an arbitrary $f \in C_c^\infty(\mathbb{R}^d)$ (smooth, compactly supported test function) and derive the PDE in the *weak* (distributional) sense.

Step 1 (Itô's formula). Since f is C^2 and X_t satisfies the SDE $dX_t = u_t(X_t) dt + \sigma_t dW_t$, Itô's formula gives:

$$df(X_t) = \nabla f(X_t)^T dX_t + \frac{1}{2} \sum_{i,j=1}^d \frac{\partial^2 f}{\partial x_i \partial x_j}(X_t) [dX_t]_i [dX_t]_j.$$

The quadratic variation of $dX_t = u_t dt + \sigma_t dW_t$ is $[dX_t^i][dX_t^j] = \sigma_t^2 \delta_{ij} dt$ (since σ_t is scalar and $dt dt = dt dW = 0$, $dW^i dW^j = \delta_{ij} dt$). Therefore:

$$df(X_t) = \underbrace{\nabla f(X_t)^T u_t(X_t) dt}_{\text{drift}} + \underbrace{\sigma_t \nabla f(X_t)^T dW_t}_{\text{martingale}} + \underbrace{\frac{\sigma_t^2}{2} \Delta f(X_t) dt}_{\text{Itô correction}}.$$

Proof of the Fokker–Planck Equation (Necessity, cont.)

Step 2a (Integrate Step 1). Integrate the Itô formula from t to $t + h$:

$$\begin{aligned} f(X_{t+h}) - f(X_t) &= \int_t^{t+h} \nabla f(X_s)^T u_s(X_s) \, ds \\ &\quad + \int_t^{t+h} \sigma_s \nabla f(X_s)^T \, dW_s \\ &\quad + \int_t^{t+h} \frac{\sigma_s^2}{2} \Delta f(X_s) \, ds. \end{aligned}$$

Step 2b (Take expectation). The three terms are handled as follows:

- *Terms 1 & 3* (Lebesgue integrals): swap $\mathbb{E}$ and $\int ds$ by Fubini–Tonelli, justified by the boundedness of ∇f , Δf (since $f \in C_c^\infty$) and the moment bound $\mathbb{E}[\sup_s \|X_s\|^2] < \infty$
- *Term 2* (Itô integral): since $f \in C_c^\infty$, the integrand $\sigma_s \nabla f(X_s)$ is bounded, so the Itô integral is a true martingale. Hence $\mathbb{E}[\int_t^{t+h} \sigma_s \nabla f(X_s)^T \, dW_s] = 0$

Proof of the Fokker–Planck Equation (Necessity, cont.)

Step 2b (cont.) After taking expectations, we are left with:

$$\mathbb{E}[f(X_{t+h})] - \mathbb{E}[f(X_t)] = \int_t^{t+h} \mathbb{E} \left[\nabla f(X_s)^T u_s(X_s) + \frac{\sigma_s^2}{2} \Delta f(X_s) \right] ds.$$

Step 2c (Differentiate). Divide both sides by $h > 0$ and let $h \rightarrow 0$. By the fundamental theorem of calculus:

$$\frac{d}{dt} \mathbb{E}[f(X_t)] = \mathbb{E} \left[\nabla f(X_t)^T u_t(X_t) + \frac{\sigma_t^2}{2} \Delta f(X_t) \right]. \quad (4.31)$$

Proof of the Fokker–Planck Equation (Necessity, cont.)

Step 3 (Rewrite using the density p_t). Since X_t has density p_t , every expectation can be written as an integral against p_t . Equation (4.31) becomes:

$$\frac{d}{dt} \int_{\mathbb{R}^d} f(x) p_t(x) dx = \int_{\mathbb{R}^d} \left[\nabla f(x)^T u_t(x) + \frac{\sigma_t^2}{2} \Delta f(x) \right] p_t(x) dx.$$

We pass $\frac{d}{dt}$ under the integral on the LHS. This is justified by dominated convergence: f has compact support and hence

$|f(x) \partial_t p_t(x)| \leq \|f\|_\infty \sup_{s \in [0,1]} |\partial_t p_s(x)|$, which is integrable by our smoothness assumption on p_t . We obtain:

$$\int_{\mathbb{R}^d} f(x) \partial_t p_t(x) dx = \int_{\mathbb{R}^d} \nabla f(x)^T (p_t(x) u_t(x)) dx + \int_{\mathbb{R}^d} \frac{\sigma_t^2}{2} \Delta f(x) p_t(x) dx. \quad (4.32)$$

Proof of the Fokker–Planck Equation (Necessity, cont.)

Step 4 (Integration by parts — transfer derivatives from f to p_t).

Recall: the multivariable integration-by-parts formula (a consequence of the divergence theorem) states: for smooth $\varphi : \mathbb{R}^d \rightarrow \mathbb{R}$ and $V : \mathbb{R}^d \rightarrow \mathbb{R}^d$ with φ compactly supported,

$$\int_{\mathbb{R}^d} \nabla \varphi \cdot V \, dx = - \int_{\mathbb{R}^d} \varphi \operatorname{div}(V) \, dx. \quad (4.33)$$

No boundary term appears because φ vanishes outside a compact set.

First term of (4.32). Apply (4.33) with $\varphi = f$ and $V = p_t u_t$:

$$\int_{\mathbb{R}^d} \nabla f(x)^T (p_t u_t)(x) \, dx = - \int_{\mathbb{R}^d} f(x) \operatorname{div}(p_t u_t)(x) \, dx. \quad (4.34)$$

Proof of the Fokker–Planck Equation (Necessity, cont.)

Second term of (4.32). We need to move both derivatives from Δf onto p_t . Since $\Delta f = \sum_{i=1}^d \partial_{x_i}^2 f$, it suffices to handle each coordinate and sum. We integrate by parts *twice*, one derivative at a time.

1st integration by parts. Apply (4.33) with $\varphi = f$ and $V = p_t \nabla f$ coordinate-wise. Since $\Delta f = \operatorname{div}(\nabla f)$:

$$\int_{\mathbb{R}^d} (\Delta f) p_t \, dx = \int_{\mathbb{R}^d} \operatorname{div}(\nabla f) p_t \, dx = - \int_{\mathbb{R}^d} \nabla f \cdot \nabla p_t \, dx.$$

(Here (4.33) is applied with $\varphi = p_t$ and $V = \nabla f$; the boundary term vanishes since ∇f is compactly supported.)

2nd integration by parts. Apply (4.33) again, now with $\varphi = f$ and $V = \nabla p_t$:

$$- \int_{\mathbb{R}^d} \nabla f \cdot \nabla p_t \, dx = + \int_{\mathbb{R}^d} f \operatorname{div}(\nabla p_t) \, dx = + \int_{\mathbb{R}^d} f \Delta p_t \, dx.$$

Combining: $\int_{\mathbb{R}^d} (\Delta f) p_t \, dx = \int_{\mathbb{R}^d} f \Delta p_t \, dx.$

Proof of the Fokker–Planck Equation (Necessity, cont.)

Step 4 (Conclusion). Substituting (4.34) and the result for the second term into (4.32):

$$\underbrace{\int_{\mathbb{R}^d} f \partial_t p_t \, dx}_{\text{LHS of (4.32)}} = - \underbrace{\int_{\mathbb{R}^d} f \operatorname{div}(p_t u_t) \, dx}_{\text{1st term after IBP}} + \underbrace{\frac{\sigma_t^2}{2} \int_{\mathbb{R}^d} f \Delta p_t \, dx}_{\text{2nd term after IBP}}.$$

Moving everything to one side:

$$\int_{\mathbb{R}^d} f(x) \left[\partial_t p_t(x) + \operatorname{div}(p_t u_t)(x) - \frac{\sigma_t^2}{2} \Delta p_t(x) \right] dx = 0.$$

This holds for every test function $f \in C_c^\infty(\mathbb{R}^d)$.

Proof of the Fokker–Planck Equation (Necessity, cont.)

Step 5 (Conclude). We have shown that for every test function $f \in C_c^\infty(\mathbb{R}^d)$:

$$\int_{\mathbb{R}^d} f(x) \left[\partial_t p_t(x) + \operatorname{div}(p_t u_t)(x) - \frac{\sigma_t^2}{2} \Delta p_t(x) \right] dx = 0.$$

By the **fundamental lemma of the calculus of variations** (du Bois-Reymond): if a continuous function integrates to zero against every $f \in C_c^\infty$, it must be identically zero. Therefore:

$$\partial_t p_t(x) = -\operatorname{div}(p_t u_t)(x) + \frac{\sigma_t^2}{2} \Delta p_t(x) \quad \text{for all } x \in \mathbb{R}^d, \ t \in [0, 1]. \quad \square$$

Proof of the Fokker–Planck Equation (Sufficiency)

Claim: If a smooth probability density p_t satisfies (4.30) with $p_0 = p_{\text{init}}$, then $X_t \sim p_t$ for all $t \in [0, 1]$.

Proof. Let q_t denote the density of X_t . By the necessity direction, q_t satisfies FP with $q_0 = p_{\text{init}}$. By hypothesis, p_t satisfies the same PDE with the same initial condition. Since FP is a linear parabolic PDE (with bounded coefficients), it has a unique solution^{15,16}. Therefore $p_t = q_t$ for all t . $\square$

¹⁵Friedman, A. (1964). *Partial Differential Equations of Parabolic Type*. Prentice-Hall, Ch. 2.

¹⁶Evans, L.C. (2010). *Partial Differential Equations*. 2nd ed., AMS, Ch. 7, §2.3.

From Flow Models to Diffusion Models

Where we stand. The Marginalization Trick (Theorem 4.10) gave us an ODE — a *flow model* — whose trajectories follow the marginal path p_t from p_{init} to p_{data} :

$$dX_t = u_t^{\text{target}}(X_t) dt.$$

The question. Suppose we want to build a *diffusion model* instead: we add Brownian noise $\sigma_t dW_t$ to the dynamics. The noise perturbs the trajectories, so X_t would no longer follow p_t . *How must we modify the drift to compensate, so that $X_t \sim p_t$ is preserved?*

The SDE Extension Trick

Theorem 4.15 (SDE extension trick)

Let $u_t^{\text{target}}(x)$ be the marginal vector field from Theorem 4.10, whose ODE generates the marginal path p_t . Then for any diffusion coefficient $\sigma_t \geq 0$, the SDE

$$X_0 \sim p_{\text{init}}, \quad dX_t = \left[u_t^{\text{target}}(X_t) + \frac{\sigma_t^2}{2} \nabla \log p_t(X_t) \right] dt + \sigma_t dW_t \quad (4.35)$$

satisfies $X_t \sim p_t$ for $0 \leq t \leq 1$. In particular, $X_1 \sim p_{\text{data}}$.

- For $\sigma_t = 0$: recovers the ODE $dX_t = u_t^{\text{target}}(X_t) dt$
- **Key insight:** Adding noise ($\sigma_t > 0$) requires a score-based correction $\frac{\sigma_t^2}{2} \nabla \log p_t$ to the drift in order to preserve the same marginal path p_t

Proof of the SDE Extension Trick

What we need to show. The SDE (4.35) has drift

$\tilde{u}_t(x) = u_t^{\text{target}}(x) + \frac{\sigma_t^2}{2} \nabla \log p_t(x)$ and diffusion σ_t . By Theorem 4.14 (sufficiency), it suffices to verify that p_t satisfies the Fokker–Planck equation with this drift:

$$\partial_t p_t(x) = -\text{div}(p_t \tilde{u}_t)(x) + \frac{\sigma_t^2}{2} \Delta p_t(x). \quad (4.36)$$

Key identity. Before starting, we record one identity that connects the score to the Laplacian. Wherever $p_t(x) > 0$:

$$\nabla p_t = p_t \nabla \log p_t \quad \Longleftrightarrow \quad \text{div}(p_t \nabla \log p_t) = \Delta p_t. \quad (4.37)$$

The left equality is the chain rule ($\nabla \log p_t = \nabla p_t / p_t$). The right follows by substituting: $\text{div}(p_t \nabla \log p_t) = \text{div}(\nabla p_t) = \Delta p_t$.

Proof of the SDE Extension Trick (cont.)

Verification. We start from what we *know* and arrive at (4.36).

By the Marginalization Trick (Theorem 4.10), u_t^{target} generates the marginal path p_t via the *continuity equation* (i.e. Fokker–Planck with $\sigma = 0$):

$$\partial_t p_t(x) = -\text{div}(p_t u_t^{\text{target}})(x). \quad (4.38)$$

Now expand the RHS of the target equation (4.36):

$$\begin{aligned} \text{RHS of (4.36)} &= -\text{div}\left(p_t \left[u_t^{\text{target}} + \frac{\sigma_t^2}{2} \nabla \log p_t\right]\right)(x) + \frac{\sigma_t^2}{2} \Delta p_t(x) \\ &= -\text{div}(p_t u_t^{\text{target}})(x) - \underbrace{\frac{\sigma_t^2}{2} \text{div}(p_t \nabla \log p_t)(x)}_{= \Delta p_t(x) \text{ by (4.37)}} + \frac{\sigma_t^2}{2} \Delta p_t(x) \\ &= -\text{div}(p_t u_t^{\text{target}})(x) \\ &= \partial_t p_t(x). \quad (\text{by (4.38)}) \end{aligned}$$

So p_t satisfies (4.36). Since $p_0 = p_{\text{init}}$, Theorem 4.14 (sufficiency) gives $X_t \sim p_t$ for all $t \in [0, 1]$. $\square$

The SDE Extension Trick — Why We Need a Conditional Version

The problem with the marginal version. Theorem 4.15 tells us the correct SDE drift is $u_t^{\text{target}}(x) + \frac{\sigma_t^2}{2} \nabla \log p_t(x)$. But *neither* term is tractable:

- The marginal vector field $u_t^{\text{target}}(x) = \int u_t^{\text{target}}(x | z) \frac{p_t(x|z) p_{\text{data}}(z)}{p_t(x)} dz$ requires integrating over p_{data}
- The marginal score $\nabla \log p_t(x)$ requires evaluating the marginal density $p_t(x) = \int p_t(x | z) p_{\text{data}}(z) dz$

Both integrals are intractable. For *training*, we need a version involving only *conditional* quantities, which we can evaluate in closed form.

The SDE Extension Trick — Conditional Version

Conditional SDE Extension Trick. For each fixed data point z , the SDE

$$dX_t = \left[u_t^{\text{target}}(X_t | z) + \frac{\sigma_t^2}{2} \nabla \log p_t(X_t | z) \right] dt + \sigma_t dW_t, \quad X_0 \sim p_{\text{init}},$$

satisfies $X_t \sim p_t(\cdot | z)$. The proof is identical: apply Fokker–Planck to $p_t(\cdot | z)$ in place of p_t , using the fact that $u_t^{\text{target}}(\cdot | z)$ generates $p_t(\cdot | z)$ via the continuity equation.

For Gaussian paths $p_t(\cdot | z) = \mathcal{N}(\alpha_t z, \beta_t^2 I_d)$, both quantities are **known in closed form** (equations (4.23) and (4.29)):

$$u_t^{\text{target}}(x | z) = \left(\dot{\alpha}_t - \frac{\dot{\beta}_t}{\beta_t} \alpha_t \right) z + \frac{\dot{\beta}_t}{\beta_t} x, \quad \nabla \log p_t(x | z) = -\frac{x - \alpha_t z}{\beta_t^2}.$$

These are the training targets.

Langevin Dynamics as a Special Case

Setup. Consider the SDE Extension Trick (Theorem 4.15) in a degenerate case: the probability path is *static*, i.e. $p_t = p$ for all t .

- Since the distribution is not changing, no transport is needed: $u_t^{\text{target}} = 0$
- (Check: the continuity equation $\partial_t p = -\text{div}(p u^{\text{target}})$ is satisfied trivially)

The SDE (4.35) reduces to:

$$dX_t = \frac{\sigma_t^2}{2} \nabla \log p(X_t) dt + \sigma_t dW_t. \quad (4.39)$$

This is the **Langevin SDE**. By the extension trick: if $X_0 \sim p$, then $X_t \sim p$ for all $t \geq 0$. In other words, p is a **stationary distribution** of the SDE.

Convergence to equilibrium. Even if $X_0 \not\sim p$, under mild conditions $X_t \rightarrow p$ as $t \rightarrow \infty$. So Langevin dynamics provides a way to *sample* from any distribution p , given only access to its score $\nabla \log p$.

Langevin Dynamics vs. Generative Modelling

Sampling from a known density. Langevin dynamics (4.39) solves the following problem: given a density p that you can *evaluate* (up to a normalising constant), produce samples $X \sim p$ by running the SDE using $\nabla \log p$.

This is a classical task in computational statistics (MCMC). Crucially, it assumes *analytic access* to p — you can compute $\nabla \log p(x)$ at any query point x .

Generative modelling is a different problem. We do *not* have access to the density p_{data} . We only have a finite dataset $z_1, \dots, z_N \sim p_{\text{data}}$. We cannot evaluate $\nabla \log p_{\text{data}}(x)$ — we do not even know p_{data} as a formula.

The bridge. This is precisely why we built the machinery of conditional probability paths and the Marginalization Trick: they reduce the problem to *conditional* quantities $u_t^{\text{target}}(x | z)$ and $\nabla \log p_t(x | z)$ that are known in closed form and can be evaluated given a single data point z from the dataset. A neural network is then trained to approximate them.

From Langevin Dynamics to Diffusion Models

Langevin as a generative model? Early score-based models¹⁷ attempted to bridge the gap: learn $\nabla \log p_{\text{data}}$ with a neural network (trained via *score matching*), then run Langevin dynamics to generate samples.

The limitation. Langevin targets a *static* distribution. Convergence to equilibrium can be very slow when p_{data} is concentrated on a low-dimensional manifold, as is typical for images.

The solution. Instead of waiting for a single static SDE to converge, *transport* the distribution along a time-varying path p_t from p_{init} to p_{data} in finite time $t \in [0, 1]$. This is exactly what the SDE Extension Trick does with $u_t^{\text{target}} \neq 0$:

$$dX_t = \left[\underbrace{u_t^{\text{target}}(X_t)}_{\text{transport}} + \underbrace{\frac{\sigma_t^2}{2} \nabla \log p_t(X_t)}_{\text{score correction}} \right] dt + \sigma_t dW_t.$$

Langevin dynamics is the special case $u_t^{\text{target}} = 0$. Diffusion models¹⁸ use both terms.

¹⁷Song, Y. & Ermon, S. (2019). Generative modeling by estimating gradients of the data distribution. *NeurIPS 2019*.

¹⁸Song, Y., Sohl-Dickstein, J., Kingma, D.P., Kumar, A., Ermon, S. & Poole, B. (2021). Score-based generative modeling through stochastic differential equations. *ICLR 2021*.

Summary: Constructing the Training Target (1/2)

Step 1: Conditional probability path. Choose a family $p_t(x | z)$ that interpolates between noise and data, one data point at a time:

$$p_0(\cdot | z) = p_{\text{init}} = \mathcal{N}(0, I_d) \quad \xrightarrow{t \in [0,1]} \quad p_1(\cdot | z) = \delta_z.$$

For Gaussian paths: $p_t(\cdot | z) = \mathcal{N}(\alpha_t z, \beta_t^2 I_d)$, with schedulers $\alpha_0 = \beta_1 = 0$, $\alpha_1 = \beta_0 = 1$.

Step 2: Conditional vector field. Find a vector field $u_t^{\text{target}}(x | z)$ whose ODE transports $p_0(\cdot | z)$ to $p_1(\cdot | z)$. For Gaussian paths (equation (4.23)):

$$u_t^{\text{target}}(x | z) = \left(\dot{\alpha}_t - \frac{\dot{\beta}_t}{\beta_t} \alpha_t \right) z + \frac{\dot{\beta}_t}{\beta_t} x.$$

Step 3: Marginal vector field. The Marginalization Trick (Theorem 4.10) shows that the mixture

$$u_t^{\text{target}}(x) = \mathbb{E}_{z \sim p_{\text{data}}} \left[u_t^{\text{target}}(x | z) \frac{p_t(x | z)}{p_t(x)} \right]$$

transports p_{init} to p_{data} via the ODE $dX_t = u_t^{\text{target}}(X_t) dt$. This is a *flow model*.

Summary: Constructing the Training Target (2/2)

Step 4: SDE Extension Trick. For any $\sigma_t \geq 0$, adding noise while correcting the drift with the score preserves the same marginal path (Theorem 4.15):

$$dX_t = \left[\underbrace{u_t^{\text{target}}(X_t)}_{\text{flow model drift}} + \underbrace{\frac{\sigma_t^2}{2} \nabla \log p_t(X_t)}_{\text{score correction}} \right] dt + \sigma_t dW_t.$$

Setting $\sigma_t = 0$ recovers the flow model. Setting $\sigma_t > 0$ gives a *diffusion model*.

Step 5: Conditional version for training. The marginal quantities $u_t^{\text{target}}(x)$ and $\nabla \log p_t(x)$ are intractable. But their conditional counterparts are known in closed form for Gaussian paths (equations (4.23) and (4.29)):

$$u_t^{\text{target}}(x | z) = \left(\dot{\alpha}_t - \frac{\dot{\beta}_t}{\beta_t} \alpha_t \right) z + \frac{\dot{\beta}_t}{\beta_t} x, \quad \nabla \log p_t(x | z) = -\frac{x - \alpha_t z}{\beta_t^2}.$$

These are the targets that a neural network $u_t^\theta(x)$ is trained to approximate.

End of Week 4